Optimization Techniques

Implementation mechanisms, dataflow, fusion, ownership and numerical contracts.

Catalogue	Implementation coverage
130 techniques / 10 areas	All 64 runtime modules and all 52 prior audited items
Production and alternatives	Experimental, historical and retired techniques retained

Updated 26 October 2026. Technique explanations are separated from measurement reports. Source links and a hash manifest make the implementation scope reviewable.

`Frozen ComfyUI: d49e888586dd8ae012c0667b33466b815fee07f7`

Kitchen 0.2.37 / TileLang 0.1.13. No inference environment changes.

Contents

Contents 2

Reading the technique catalogue 3

SYS / Partitioning and persistent execution 4

LIN / INT8 projections, fusion and communication 7

NUM / Numerical storage and lifetime management 12

ATT / Dense attention and preparation techniques 14

SPR / Sparse routing and pooled attention 20

LOR / Online adapter ownership and arithmetic 24

IO / Offline baking and checkpoint access 27

RES / Host cache, GPU windows and phase scheduling 31

VAE / Component loading and parallel decode 36

ALT / Retained alternative optimization methods 40

Core equations 44

Implementation source catalogue 45

Reading the technique catalogue

Each entry gives a mechanism, the work or storage removed, and its numerical and ownership contract. Implementation paths refer to the repository. Selection labels distinguish active code from alternatives, experiments and retired methods.

M denotes activation rows; K and N are projection input/output widths; H and Hkv are query/KV heads; D is head dimension; r is the TP owner. SUM and MAX are distributed collectives. A view retains its backing allocation.

Boundary	Current selection	Scope
TE attention	kitchen_causal	Pinned masked Kitchen INT8; compact GQA
DiT / refiner	kitchen / Native SDPA	Sparse modes remain explicit
Columns	cuda_int8_tilelang	Guarded aligned SM75 DiT shapes; preallocated fallback
FC2 / down preparation	fused_stored	Explicit fused_recompute and owned_fp32 alternatives
Row communication	tilelang_scaled_fp16	Explicit INT32 SUM alternative
Tensor storage	FP16-fix; divisor 64	FP32 numerical islands; public unscaled outputs
LoRA	offline	Optional ordered online adapters with FP32 Row-A SUM
Scheduling	Lifecycle / pressure / adaptive enabled	Idle residency remains explicit
Video / Audio	Query-parallel Kitchen / Native FP32	Coordinated Native Video fallback

Numerical contracts are part of the technique. INT32 SUM, scaled FP16 SUM, Group32, Kitchen INT8 and Native SDPA must not be conflated. A returned FP32 dtype can contain values previously rounded to BF16.

Technique coverage includes retained implementations without claiming that every candidate is selected. Benchmark tables, failure narratives and experiment chronology remain in separate reports.

SYS / Partitioning and persistent execution

Keep Native conditioning, sampling and public interfaces in the parent. Partition learned computation inside persistent workers. Component ownership, collective order and resource ownership are parts of the algorithm.



Conceptual dataflow and ownership; independent techniques have their own contracts.

SYS01 Component-aware tensor parallelism

Selection: active

Split column-parallel weight output rows and row-parallel input columns. For packed QKV and gate/up tensors, split each component independently and concatenate the owned pieces in Native component order.

Work removed. Each rank stores and multiplies its owned matrix pieces instead of replicating the complete TE and DiT block.

Contract. Replicate the required scales, norms and metadata. A half of a packed tensor is not generally the same as half of each component.

`Implementation: patches/rank_local_int8.py; patches/tp_qwen3vl.py; patches/tp_dit.py`

SYS02 Persistent serial worker sessions

Selection: active

Create one spawned process per CUDA device and reuse its initialized modules, readers and caches across commands. Carry sequence identifiers through a serial request/acknowledgement protocol.

Work removed. Amortize process, CUDA and module startup and retain useful prepared weights between actual evaluations.

Contract. Preserve collective ordering and propagate failures before reusing a session. Persistence does not imply concurrent sampling requests.

`Implementation: patches/comfy_tp_runtime.py; patches/comfy_sampler.py`

SYS03 Bounded CPU packet handoff

Selection: active

Serialize the required tensors and nested layout metadata through the parent/worker protocol at explicit phase boundaries. Preserve dtype and PackedLayout rather than reconstructing context from tensor size.

Work removed. Keep Native orchestration outside GPU workers while avoiding repeated model construction.

Contract. Handoff is synchronous. Do not infer communication/compute overlap from the existence of persistent processes.

`Implementation: patches/comfy_tp_runtime.py; patches/comfy_sampler.py`

SYS04 Discard unused secondary-rank replies

Selection: active

After both ranks finish computation and synchronization, discard rank-1 nondiagnostic output before making the CPU packet. Keep acknowledgements and explicit diagnostic responses.

Work removed. Remove a redundant device-to-host tensor copy, CPU allocation and pipe payload.

Contract. Both ranks still participate in all collectives and error handling; output suppression is not computation suppression.

`Implementation: patches/comfy_tp_runtime.py`

SYS05 Stable collective topology

Selection: active

Use NCCL with peer-to-peer disabled and four minimum/maximum channels; retain automatic protocol selection. Run corresponding collectives in the same order on both owners.

Work removed. Bound the communication configuration for the two-device deployment and avoid depending on unavailable peer-copy topology.

Contract. Actual distributed collectives remain necessary. Logical-rank execution on one GPU is not a replacement for this topology.

`Implementation: patches/comfy_tp_runtime.py; patches/quantized_collectives.py`

SYS06 Policy-keyed prepared module reuse

Selection: active

Attach an immutable operator and numerical-flow policy to prepared modules, streamed specifications and cache identities. Recreate the service when loader choices change.

Work removed. Reuse prepared representations without applying a second fold or serving weights prepared under a different backend or LoRA stack.

Contract. Residency copies must preserve registered buffers, rank metadata and policy; avoid methods bound to the canonical CPU module.

`Implementation: patches/h3_operator_policy.py; patches/comfy_tp_runtime.py; patches/weight_residency.py`

SYS07 Architecture and selected-backend guards

Selection: active

Select specialized kernels only for supported inference geometry and SM75. Retain the selected public Kitchen implementation for other backends, autograd, compiled calls, hooks and unsupported shapes.

Work removed. Concentrate specialized code on the H3 shapes it implements without replacing the global operator registry.

Contract. Validate the pinned core ABI before execution. A fallback must preserve the public shape, dtype and mask interface.

`Implementation: patches/h3_operator_dependencies.py; patches/legacy_runtime.py; patches/h3_configuration.py; custom_nodes/minimax_h3_tp2/__init__.py`

SYS08 Separate hot-path execution from diagnostics

Selection: active

Keep detailed events, tensor scans, observers and profiler capture behind explicit diagnostic entry points. Normal worker commands retain only required protocol and lifecycle accounting.

Work removed. Avoid tracing allocation, host synchronization and repeated scalar extraction during ordinary inference.

Contract. Required acknowledgements, failures and admission decisions remain available. Diagnostics must not become an implicit production dependency.

`Implementation: patches/diagnostic_metrics.py; patches/runtime_diagnostics.py; patches/comfy_tp_runtime.py`

SYS09 Full-model backend geometry at sharded boundaries

Selection: active

When Native attention backend priority depends on the original query size, reconstruct that size from the head shard and world size before selecting the retained backend policy.

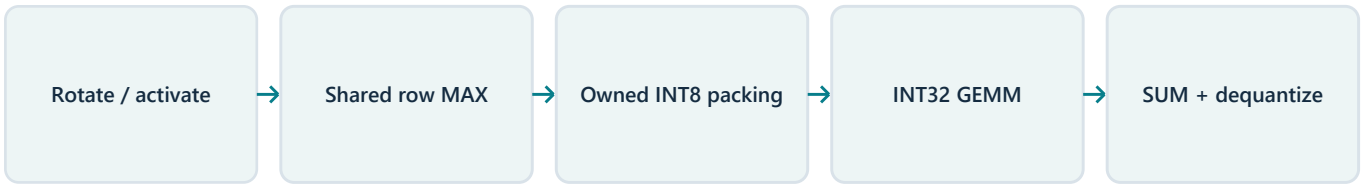
Work removed. Keep tensor parallelism from accidentally changing a shape-dependent Native kernel selection.

Contract. Preserve the complete model threshold and supported public attention API even though each rank owns fewer heads.

`Implementation: patches/tp_qwen3vl.py`

LIN / INT8 projections, fusion and communication

A coherent TP row operator uses a shared activation quantizer, local INT8 GEMMs, a collective on partial results and a deliberate dequantization order. Fusion removes materialized intermediates while preserving the selected arithmetic boundaries.



Conceptual dataflow and ownership; independent techniques have their own contracts.

LIN01 Direct Turing INT8 matrix multiplication

Selection: active

Call the retained Kitchen CUDA/cuBLAS INT8 matrix primitive on contiguous activation codes and owned weight rows, producing an INT32 accumulator.

Work removed. Avoid eager per-call transposed-weight and padded-row intermediates where the scoped direct path applies.

Contract. Keep the complete intended GEMM geometry, signed integer operands and original activation/weight scales.

`Implementation: patches/optimized_tp.py; patches/parallel_int8.py`

LIN02 Global-row activation scale across owners

Selection: active

Compute local maxima over the owned rotated activation slice, then MAX-reduce them. Quantize both owners with the same full-row maximum before local GEMMs.

Work removed. Avoid gathering the entire floating activation merely to reproduce a full-row quantizer.

Contract. Independent rank-local scales cannot be followed by a single common INT32 SUM/dequantization. Preserve the shared scale contract.

`Implementation: patches/optimized_tp.py; patches/tilelang_row_quant.py`

LIN03 External-maximum quantization

Selection: active

Pass the already reduced full-row maximum directly into the owned-slice quantizer. Apply the pinned division, nearest rounding, clamp and scale floor in the packing kernel.

Work removed. Remove an artificial maximum sentinel, extra scan, temporary copy and restoration stage.

Contract. Match the selected quantizer rounding and scale-floor order, including zero rows; algebraically similar division is not automatically bit-equivalent.

`Implementation: patches/tilelang_row_quant.py`

LIN04 Strided maxima and direct owned reads

Selection: active

Read each owner from the original packed or strided activation and reduce its absolute maximum in place. Use explicit source strides and ownership offsets.

Work removed. Remove full-width contiguous copies and intermediate owned-slice materialization.

Contract. Handle component packing and padding separately. Source views must remain live until the consuming kernel finishes.

`Implementation: patches/tilelang_row_quant.py`

LIN05 Positive-zero ownership padding

Selection: active

Write the owner codes directly to aligned GEMM storage and initialize every unowned/padded position as positive zero. Define padding from ConvRot groups and packed component boundaries.

Work removed. Fuse ownership packing, layout conversion and padding into one destination.

Contract. Signed zero and uninitialized tail bytes can change raw-byte equivalence or subsequent transforms. Read only valid source channels.

`Implementation: patches/tilelang_row_quant.py; patches/tilelang_swiglu_pad.py`

LIN06 SwiGLU plus ownership-padding fusion

Selection: active

Compute silu(gate) * up in the selected floating island and store directly into the padded/owned destination. Support separate TE gate/up tensors and packed DiT branch inputs.

Work removed. Remove eager activation/product tensors and a later ownership-padding copy.

Contract. Promoted FP16-flow preparation converts operands to FP32 before the nonlinear product. Converting an already overflowed product is too late.

`Implementation: patches/tilelang_swiglu_pad.py; patches/h3_operator_policy.py`

LIN07 Owned FP32 ConvRot preparation

Selection: alternative

Compute the supported rank-owned SwiGLU/ConvRot channels in FP32 and feed the common global-MAX quantization procedure. Retain complete transform groups across ownership boundaries.

Work removed. Avoid a full-width FP32 rotated activation while retaining the required local slice.

Contract. This is the owned_fp32 alternative. Its grouping and input layout guards differ from the newer regular-H4 fused producer.

`Implementation: patches/tilelang_swiglu_pad.py`

LIN08 Stored fused FP32 SwiGLU and regular H4 rotation

Selection: active

Fuse branch activation and regular ConvRot256 preparation in FP32, retain the rotated FP32 intermediate, and produce owned INT8 codes using the shared row maximum.

Work removed. Collapse separate activation, rotation and packing launches while reusing the rotated result in subsequent preparation.

Contract. Use the pinned H4/ConvRot order and explicit rounding boundaries. This is fused_stored, the default eligible FC2/down preparation.

`Implementation: patches/tilelang_swiglu_quant.py; patches/h3_operator_policy.py`

LIN09 Recomputed fused preparation

Selection: alternative

Calculate the same FP32 branch/rotation once for maxima and recompute it for the final packing rather than retaining the complete rotated activation.

Work removed. Trade additional arithmetic and reads for a shorter/smaller FP32 intermediate lifetime.

Contract. Keep the stored variant arithmetic contract. Recompute is an explicit memory alternative, not an automatic faster path.

`Implementation: patches/tilelang_swiglu_quant.py`

LIN10 Exact INT32 partial SUM

Selection: active

SUM the local INT32 GEMM accumulators before applying the common row scale and output-channel weight scale, then bias and output conversion in the required order.

Work removed. Preserve the integer reduction contract without gathering full activations.

Contract. INT32 SUM is the explicit accuracy alternative to scaled FP16 communication. The native fallback reduction is fixed, not a second hidden policy switch.

`Implementation: patches/parallel_int8.py; patches/optimized_tp.py; patches/tilelang_swiglu_pad.py`

LIN11 Power-of-two FP16 partial transport

Selection: active

Choose an exponent from the local-K accumulator bound and world size. Encode partial INT32 values as FP16 after power-of-two scaling, AllReduce SUM, and reconstruct/dequantize in FP32.

Work removed. Halve partial-sum payload size relative to INT32 and fuse reconstruction with the final epilogue.

Contract. FP16 encoding and SUM are approximate, especially under cancellation. Bound both rank contributions below FP16 maximum; scaling changes range, not mantissa bits.

`Implementation: patches/tilelang_transport_p2.py; patches/tilelang_transport_tradeoffs.py`

LIN12 Fused row dequantization epilogue

Selection: active

Combine transport reconstruction, shared row scale, weight scale, bias and final dtype store in one kernel after the collective.

Work removed. Remove full-size reconstructed and separately dequantized floating temporaries.

Contract. Keep explicit FP32 operation order and the final activation-store boundary; apply online LoRA at its defined post-base boundary.

`Implementation: patches/tilelang_transport_p2.py; patches/parallel_int8.py`

LIN13 Fixed-size feature gather fallback

Selection: fallback

Gather equal-size rank-owned feature pieces into fixed receive storage and restore Native order before the retained fallback operator.

Work removed. Provide a predictable fallback without variable-size object transport or repeated gather-list construction.

Contract. The gathered floating path is distinct from shared-scale local INT32 partials. It remains a fallback, not an INT8 communication claim.

`Implementation: patches/optimized_tp.py; patches/parallel_int8.py`

LIN14 Single-destination chunked column dequantization

Selection: active

Allocate the final output once. Process bounded accumulator row chunks, form row_scale * weight_scale in FP32, multiply the converted accumulator in place, and copy to final rows.

Work removed. Remove a list of output chunks, their final concatenation and a redundant full-size scaled intermediate.

Contract. Keep the two FP32 multiplication boundaries in this order. Chunking the epilogue does not change GEMM dimensions.

`Implementation: patches/dit_column_linear.py`

LIN15 Direct column GEMM with ordered TileLang epilogue

Selection: active

Reuse eager activation rotation/quantization, call the direct INT8 GEMM for aligned SM75 DiT QKV/FC1 weights, then apply an epilogue with explicit rounded FP32 products and one output store.

Work removed. Remove eager weight transpose/padding and broadcast dequantization scratch on the supported column shapes.

Contract. Require inference, the selected eager backend and the guarded DiT geometry. Preserve the public backend elsewhere; epilogue-only tilelang_exact remains a private comparator.

`Implementation: patches/dit_column_linear.py; patches/tilelang_column_epilogue.py`

LIN16 Shared TE input rotation and quantization

Selection: active

Prepare a common normalized input once for independent Q/K/V column GEMMs, and similarly once for gate/up. Reuse only deterministic activation codes and row scales.

Work removed. Remove repeated ConvRot and row-quantization work without merging projection GEMMs.

Contract. Keep hooks, subclasses, overridden forwards, autograd and compiled calls on their original module paths. Preserve each projection weight and GEMM geometry.

`Implementation: patches/dit_column_linear.py; patches/tp_qwen3vl.py`

LIN17 Bounded complete column-row scheduling

Selection: experimental

Partition large inference activation rows into bounded calls around the public linear API; write each completed block to its final output position.

Work removed. Bound per-call accumulator/workspace residency for long sequences.

Contract. A changed GEMM row geometry can change floating/backend behavior. This experimental schedule is distinct from epilogue-only chunking.

`Implementation: benchmarks/bounded_column_int8.py`

NUM / Numerical storage and lifetime management

Reduce activation bytes with explicit storage boundaries. Preserve FP32 statistics and nonlinear preparation where range or rounding requires them. Residual gauges are coordinate changes on privately prepared modules, not checkpoint requantization.



Conceptual dataflow and ownership; independent techniques have their own contracts.

NUM01 FP16 residual storage with a power-of-two gauge

Selection: active

Represent the residual stream as y = x / c, with c = 2^e and default e = 6 for TE/DiT. Divide branch outputs before storage and use epsilon / c^2 for gauge-aware normalization.

Work removed. Reduce residual/activation storage and movement while preserving range headroom.

Contract. Power-of-two scaling does not add precision. Vision/Video gauges remain independently specified; legacy flow retains the original coordinate system.

`Implementation: patches/h3_tensor_flow.py; patches/comfy_tp_runtime.py`

NUM02 Private scale and bias folding

Selection: active

Fold the scalar residual divisor into private FP32 INT8 weight scales or dense weights and biases once during module preparation. Adjust LoRA post-B coefficients consistently.

Work removed. Avoid separate runtime branch-rescale tensors and repeated multiplications.

Contract. Never mutate checkpoint bytes or apply a fold twice. Nonuniform gains cannot be commuted through ConvRot as a scalar fold.

`Implementation: patches/h3_tensor_flow.py`

NUM03 FP32 normalization and modulation islands

Selection: active

Compute normalization statistics, gains, epsilon, AdaLN modulation and gated additions in FP32, then cast at the declared activation storage boundary.

Work removed. Allow compact FP16 storage around operations whose reductions and products need wider range.

Contract. Keep sampler state, time curves, statistics and final DiT heads in their FP32 islands. Restore public conditioning in its unscaled representation.

`Implementation: patches/h3_tensor_flow.py; patches/tp_dit.py; patches/tp_qwen3vl.py`

NUM04 FP32 online adapter products before scaling

Selection: active

For FP16 tensor flow, perform the A/B products in FP32 and apply the adapter strength after B before the final activation store.

Work removed. Prevent an overflowing half-precision B product from destroying information before a small post-B coefficient is applied.

Contract. Preserve adapter order and strength placement; a small scale cannot recover a product that already overflowed.

`Implementation: patches/h3_lora.py; patches/h3_tensor_flow.py`

NUM05 Prompt release of attention and MLP intermediates

Selection: active

Release caller references to Q/K/V, attention buffers, branch tensors, LoRA output views and old residual owners immediately after their last consuming operation.

Work removed. Reduce overlap between attention, FFN, residual and next-block activation lifetimes.

Contract. Deleting a local name is effective only if callers and views no longer retain the storage. Respect stream/event lifetime before reuse.

`Implementation: patches/tp_qwen3vl.py; patches/tp_dit.py; patches/h3_lora.py`

NUM06 Retained fused norms and rotary embedding

Selection: active

Use the pinned Kitchen RMSNorm/RoPE helpers at the established Native boundary, including compact head layouts and required gain casts.

Work removed. Fuse normalization/rotary work and avoid otherwise separate layout and elementwise passes.

Contract. Preserve Native residual, normalization, projection and rotation ordering. This does not authorize a replacement of the frozen legacy attention API.

`Implementation: patches/tp_dit.py; patches/tp_qwen3vl.py; patches/h3_operator_policy.py`

NUM07 BF16 reference activation storage

Selection: experimental

Use the numerical-flow wrapper with BF16 storage while retaining the same explicit FP32 stability islands and private residual-gauge preparation.

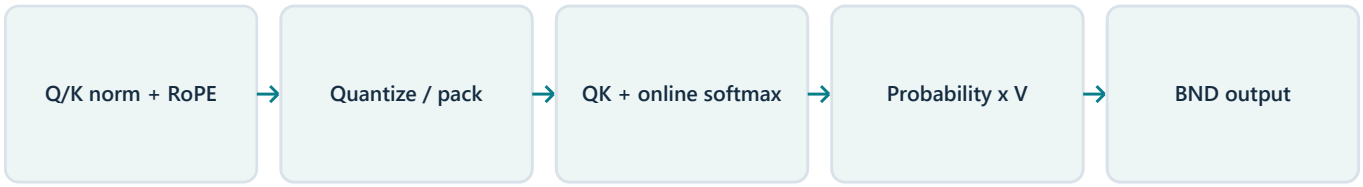
Work removed. Provide a 16-bit storage/reference representation with a different range/precision tradeoff from FP16.

Contract. This is reference-only; deployment RuntimeConfig rejects bf16_reference on the T4 path. Storage support does not imply native BF16 Tensor Core arithmetic on SM75.

`Implementation: patches/h3_tensor_flow.py; patches/comfy_tp_runtime.py`

ATT / Dense attention and preparation techniques

Dense attention techniques change operand preparation, reuse, layout or scheduling. State the quantization and output-store contract before discussing fusion: INT8, Group32 and Native SDPA are distinct numerical paths.



Conceptual dataflow and ownership; independent techniques have their own contracts.

ATT01 Pinned Kitchen INT8 dense attention

Selection: active

Rotate/quantize Q/K, use packed per-thread scales and an optional representative-key shift, and accumulate attention with the retained signed-INT8 Q/K and unsigned-probability/INT8 V primitives.

Work removed. Use Tensor Core integer products and online softmax without materializing a token-by-token score matrix.

Contract. Preserve the installed quantizer and scale packing. FP32 input uses BF16 output storage followed by an FP32 cast, not full FP32 numerical precision.

`Implementation: patches/h3_operator_policy.py; experiments/kitchen_attention/attention.py; experiments/kitchen_attention/native/vendor/attn_utils.cuh`

ATT02 Direct masked Kitchen TE with compact GQA

Selection: active

Forward the original boolean/additive mask and compact KV heads directly to pinned Kitchen, then expose the BND output expected by TE.

Work removed. Avoid KV-head replication, full-mask reconstruction, host mask classification and a separate native-extension build in default dispatch.

Contract. Score masking and raw-input prefix invariance are separate contracts: shared data-dependent quantization can couple earlier representations to future inputs.

`Implementation: patches/kitchen_te_attention.py; patches/h3_operator_policy.py`

ATT03 Native SDPA for the token refiner

Selection: active

Keep the Native refiner projections, normalization, rotary embedding and SDPA while wrapping the module so policy survives residency copies.

Work removed. Avoid applying sparse/quantized main-DiT attention preparation to the small refiner sequence.

Contract. Refiner tokens are not sparsified. Use a registered module wrapper rather than a closure bound to a CPU canonical object.

`Implementation: patches/h3_operator_policy.py`

ATT04 Group32 QK with FP16 PV

Selection: alternative

Partition each head dimension into 32-channel groups. Compute an INT32 QK product per group, apply the corresponding FP32 scales, and use FP16 probabilities/values for PV.

Work removed. Use narrower quantization groups and Tensor Core QK while retaining a wider PV path.

Contract. This is an explicit numerical alternative to Kitchen per-thread packing. Preserve the centering and rounding selected by its preparation.

`Implementation: patches/tilelang_grouped_qk.py; patches/tilelang_grouped_tp.py; patches/tilelang_grouped_tp_kitchen.py`

ATT05 Mask-aware Group32 TE kernel

Selection: alternative

Broadcast compact 2D/3D/4D masks within online softmax and map query heads to shared KV heads without expanding K/V. Apply additive bias in the kernel score domain.

Work removed. Avoid repeated KV copies and full expanded mask storage for GQA prefill.

Contract. Preserve empty-row handling and later-leaf accumulation after masked leaves. tilelang_group32 is an explicit override, not the current TE default.

`Implementation: patches/tilelang_group32_masked.py`

ATT06 Fused stride-aware cast, pad and quantize

Selection: alternative

Read packed/strided Q/K/V directly and combine input conversion, ragged-tail padding, Group32 scale reduction and code production. Keep only mode-required representations.

Work removed. Remove separate contiguous/cast/pad passes and avoid duplicate padded floating tensors.

Contract. Use the selected input FP16 rounding boundary, nearest rounding and scale floor. Arbitrary layout fusion must retain source strides and valid tails.

`Implementation: patches/tilelang_grouped_tp_kitchen_v2.py; patches/tilelang_grouped_fused.py`

ATT07 Prepared dense-only fast path

Selection: experimental

Build an internally owned all-dense route object and prepare only dense operands. Omit pooled V, V8 and LSE work when the consumer does not read them.

Work removed. Remove unused routing metadata and repeated GPU-to-CPU validation for internally generated immutable routes.

Contract. Only the exact internally generated route object may use the prevalidated path. Older GQA preparation may explicitly expand heads and must not be confused with compact GQA.

`Implementation: patches/tilelang_dense_preparation.py; patches/tilelang_sage_preparation.py`

ATT08 Bounded immutable geometry reuse

Selection: alternative

Cache counts, padding geometry and all-dense metadata by device, stream and tensor geometry; keep content-dependent Q/K/V and routes outside immutable caches.

Work removed. Remove repeated allocation and geometry kernel launches on recurring shapes.

Contract. Bound cache size and distinguish stream ownership. Reusing immutable metadata does not make activation-dependent summaries reusable.

`Implementation: patches/tilelang_grouped_tp_optimized.py; patches/tilelang_dense_preparation.py`

ATT09 Query-only and full-QK reuse

Selection: alternative

Retain query fragments across key tiles; the both-cache candidate additionally retains key fragments when its kernel mapping permits. The selected optional dense path uses query-only reuse.

Work removed. Reduce repeated shared/global operand loads at the cost of longer register lifetimes.

Contract. Cache choice interacts with head dimension, query tile, scale fragments and occupancy. A both-cache flag is a separate candidate, not implied by query-only selection.

`Implementation: patches/tilelang_grouped_tp_kitchen_v2.py; patches/tilelang_attention_tradeoffs.py`

ATT10 Warp, query-tile and scale-placement specialization

Selection: experimental

Specialize BQ32/BQ64 schedules, 128/256-thread blocks, row/square warp mappings, register or staged scales and row-score fragments for fixed geometry.

Work removed. Trade reuse, parallel rows, register pressure and staging traffic without changing the intended attention equation.

Contract. Every specialization must keep valid tail/mask handling. Exposed candidate flags do not mean the production caller selects all of them.

`Implementation: patches/tilelang_grouped_tp_kitchen_v2.py; patches/tilelang_attention_tradeoffs.py`

ATT11 Direct BND and dtype epilogue stores

Selection: alternative

Map output fragments directly to batch-token-flattened-head storage and perform the declared dtype conversion in the kernel epilogue.

Work removed. Remove a BHND-to-BND transpose/contiguous copy and a later cast.

Contract. Retain an intermediate FP16 rounding boundary when that is part of the original path. The finite-output epilogue is a separate guarded candidate, not permission to hide nonfinite values.

`Implementation: patches/tilelang_group32_masked.py; patches/tilelang_attention_tradeoffs.py`

ATT12 Sage-style block scales and centering alternatives

Selection: experimental

Quantize Q and K with independently selected block sizes, token-mean or leaf-mean centering, and explicit half-away or nearest-even rounding variants.

Work removed. Reduce preparation cost or adapt the quantizer granularity while retaining INT8 QK and floating PV.

Contract. Different group sizes, centering means and rounding rules define different approximations. Ragged leaf means must use actual counts.

`Implementation: patches/tilelang_sage_qk.py; patches/tilelang_sage_preparation.py`

ATT13 Prequantize then release floating operands

Selection: experimental

Separate the pinned Q/K/V quantizer from its attention consumer. Retain packed integer operands, scales, mask and launch metadata while releasing floating inputs when their callers no longer need them.

Work removed. Shorten raw Q/K/V lifetimes and expose prepared-operand reuse without rerunning quantization.

Contract. The snapshot owns a mask representation and is immutable with respect to input/mask changes. Storage/API dtype is not a promise of full FP32 precision.

`Implementation: experiments/kitchen_attention/attention.py; experiments/kitchen_attention/te_precision.py`

ATT14 Implicit causal index masking

Selection: experimental

Intersect key traversal with key_index <= query_index + query_offset, avoiding a materialized token-pair triangle in the explicit native kernel.

Work removed. Remove full causal-mask storage and mask loads in supported prefill geometry.

Contract. Keep offsets, ragged bounds and GQA mapping explicit. Implicit score masking does not make the preserved global quantizer prefix invariant.

`Implementation: experiments/kitchen_attention/native/kitchen_sm75.cu; experiments/kitchen_attention/attention.py`

ATT15 Kitchen query64 and cached-Q schedules

Selection: experimental

Derive an SM75 extension from the retained Kitchen source with a smaller query tile or a query128 schedule retaining Q fragments in registers.

Work removed. Change CTA parallelism and repeated operand loading while keeping quantization and core integer arithmetic intact.

Contract. Keep the original schedule as the control. Cached Q requires the supported D128/key128 geometry; register/spill checks guide specialization selection.

`Implementation: experiments/kitchen_attention/derive.py; experiments/kitchen_attention/native/kitchen_sm75.cu`

ATT16 n8 probability-value scheduling

Selection: experimental

Dispatch to the retained compute_int8_sv_pipelined_n8 helper for its supported D128/key128 tile. Shorten live V/partial fragments while preserving the four integer products and FP32 FMA order.

Work removed. Reduce live fragment pressure and change instruction scheduling without replacing the probability quantizer.

Contract. Use only supported warp/tile geometry. Spill elimination alone is not a performance guarantee; unsupported geometries retain the original helper.

`Implementation: experiments/kitchen_attention/native/h3_dense_utils.cuh; experiments/kitchen_attention/derive.py`

ATT17 Guarded narrow tensor addressing

Selection: experimental

Use uint32 tensor offsets for supported dense specializations only after checking the batch/head/length/dimension product against the address range in Python and CUDA dispatch.

Work removed. Reduce wide pointer-offset arithmetic and associated register lifetimes.

Contract. Reject overflow before allocation/launch and retain the uint64 implementation. Narrow addressing is not valid merely because an individual sequence length is small.

`Implementation: experiments/kitchen_attention/attention.py; experiments/kitchen_attention/native/kitchen_sm75.cu`

ATT18 Explicit fixed-shape CUDA Graph replay

Selection: experimental

Capture the unchanged Kitchen operator and BND conversion after side-stream warmup. Copy current inputs into persistent buffers for each replay and clone the output for independent ownership.

Work removed. Amortize repeated Python/driver launch setup for a fixed sequence of kernels.

Contract. Enforce creating-stream, dtype and geometry identity; retain graph-private storage and count input/output copies and capture lifetime. No production graph cache is installed.

`Implementation: experiments/kitchen_attention/graph.py`

ATT19 Mutation-aware causal mask recognition cache

Selection: experimental

Recognize only an exact causal triangle, then cache its identity and mutation version using weak references. Recheck versionless inference tensors and reject padding/custom bias from this shortcut.

Work removed. Amortize repeated classification when a normal immutable mask is explicitly routed to the native causal experiment.

Contract. The default TE path bypasses recognition. Classification can synchronize the host, and inference tensors cannot safely use a mutation-version cache.

`Implementation: patches/kitchen_te_attention.py`

ATT20 Fingerprint-keyed native build reuse

Selection: experimental

Hash the retained native/Python build inputs, serialize CPU compilation with a bounded exclusive lock, and reuse a binary only after source, compiler, architecture and artifact-hash validation.

Work removed. Avoid redundant compilation by multiple workers and accidental reuse of a stale experimental binary.

Contract. Compilation is explicit and installs no dependencies. Imports/default TE dispatch do not compile; a foreign build lock is never removed by a waiter.

`Implementation: patches/kitchen_te_attention.py; experiments/kitchen_attention/build.py`

ATT21 Group32 and Group64 scale-granularity alternatives

Selection: experimental

Choose the channel-group width for independent Q/K scales and restore each group product scale before summing logits. Keep the PV path and input/output rounding boundaries explicit.

Work removed. Trade quantization-range locality against the number of group products, scale loads and fragments.

Contract. Different group widths are separate approximations. The selected Group32 path must not be relabeled as a Group64 or whole-head result.

`Implementation: patches/tilelang_grouped_qk.py; benchmarks/attention_grouped_study.py`

ATT22 Cast/pad centering preparation

Selection: experimental

Fuse the FP16 input cast and ragged padding stage used to build key leaf means, then feed the selected mean/centering recipe into operand quantization.

Work removed. Remove a separate conversion/padding launch and duplicate preparation storage.

Contract. Preserve actual leaf counts and the recipe distinction between unweighted leaf means and token-weighted means. castpad is an explicit preparation variant.

`Implementation: patches/tilelang_grouped_tp_kitchen_v2.py`

SPR / Sparse routing and pooled attention

Route blocks of tokens, preserve protected conditioning, and explicitly define omitted attention. Sparse dropping and scalar pooling implement different approximation equations. Scheduling and preparation are independent optimization dimensions.



Conceptual dataflow and ownership; independent techniques have their own contracts.

SPR01 Leaf64 centroid routing

Selection: active

Compute query/key means over 64-token leaves, account for ragged counts, center keys and compare centroid scores to the selected threshold. Retain first/global and adjacent leaves as mandatory routes.

Work removed. Replace token-pair routing decisions with a block-sized route matrix and skip fine QK/PV for omitted blocks.

Contract. Keep the exact centering and threshold recipe for the selected variant. A routing approximation changes which token interactions are computed.

`Implementation: patches/tilelang_grouped_tp_optimized.py; patches/tilelang_sparse_attention.py`

SPR02 PackedLayout-derived protected prefix

Selection: alternative

Read the current Native packed segment layout and protect every complete leaf intersecting the prefix before the final generated-video span.

Work removed. Apply sparsity to the intended video region while retaining text, references, conditioning and generated audio interactions.

Contract. Reject stale/missing layouts or ambiguous final video spans. Do not substitute a fixed historical prefix token count.

`Implementation: patches/h3_operator_policy.py`

SPR03 Mode-specific preparation and finite tau0 shortcut

Selection: alternative

For drop mode, omit pooled/V8/LSE representations. For scalar mode, prepare its required summaries. Use the tau0 algebraic shortcut only when centered means and finite inputs satisfy its assumptions.

Work removed. Remove preparation and threshold work the chosen sparse consumer cannot use.

Contract. The shortcut is not valid for arbitrary nonfinite tensors or another centering convention. Keep dense-prefix and mandatory-route rules.

`Implementation: patches/tilelang_grouped_tp_optimized.py; patches/tilelang_sol_drop_fused.py`

SPR04 Sparse dropping

Selection: alternative

Traverse selected fine leaves and update online softmax/output only from their tokens. Renormalize over the retained interactions.

Work removed. Eliminate fine QK and PV products for omitted leaves.

Contract. Omitted mass is discarded. This is not numerically equivalent to dense attention or count-weighted pooling.

`Implementation: patches/tilelang_grouped_tp_optimized.py; patches/h3_operator_policy.py`

SPR05 Scalar pooling of omitted leaves

Selection: alternative

Represent each omitted leaf by a mean key/value and weight its exponential score by the valid token count. Merge fine and coarse mass in a shared normalization state.

Work removed. Replace many omitted-token products with one centroid contribution while retaining an approximation to their attention mass.

Contract. Use actual ragged counts and omitted-only contributions. Selected leaves must not be counted again as centroids.

`Implementation: patches/tilelang_scalar_pool.py; patches/tilelang_grouped_tp_optimized.py`

SPR06 Capped omitted-centroid score workspace

Selection: alternative

Precompute omitted centroid scores only when B * H * padded_queries * leaf_count * 4 fits the configured cap. Otherwise calculate coarse contributions inline.

Work removed. Move repeated scalar-score work out of the fine loop without permitting an unbounded query-by-centroid allocation.

Contract. A zero cap selects inline processing, not disabled sparsity. Workspace geometry includes all head owners and padding.

`Implementation: patches/tilelang_attention_p2.py; patches/h3_operator_policy.py`

SPR07 GPU ballot route compaction

Selection: experimental

Convert block masks into sorted selected-key leaf indices with per-query counts, using GPU ballot/prefix operations. Traverse only initialized active list slots.

Work removed. Avoid scanning rejected leaves inside every fine attention tile.

Contract. Keep ordering, empty-row behavior and list capacity explicit. Unused index slots are unspecified and must never be consumed.

`Implementation: experiments/kitchen_attention/native/kitchen_sm75.cu; patches/tilelang_attention_tradeoffs.py`

SPR08 Native Kitchen routed leaf execution

Selection: experimental

Use compact selected-leaf lists with the retained Kitchen quantization and INT8 QK/PV primitives. Intersect explicit routes with token causal bounds when requested.

Work removed. Skip omitted native QK/PV products while reusing Kitchen operand packing.

Contract. Routing changes scheduling to supported query64/key64 geometry. Noncausal content routes and explicit causal routes have separate contracts.

`Implementation: experiments/kitchen_attention/native/routed_loop.cuh; experiments/kitchen_attention/attention.py`

SPR09 Pooled centroids in quantized operand space

Selection: experimental

Build omitted K/V centroids after the retained rotation, centering and dequantization so fine and coarse scores share the same key-anchor shift.

Work removed. Avoid mixing incompatible score origins when combining native INT8 fine attention and FP16 coarse attention.

Contract. This is a different approximation from older Group32 scalar pooling. Coarse means and fine output rounding remain declared numerical boundaries.

`Implementation: experiments/kitchen_attention/native/kitchen_sm75.cu; experiments/kitchen_attention/attention.py`

SPR10 Tiled coarse attention and fused LSE merge

Selection: experimental

Batch centroid contributions, retain FP32 maximum/denominator state, and merge a fine output/LSE with coarse count-weighted attention directly into the final output tile.

Work removed. Avoid a full Q-half copy, full query-by-centroid score tensor, separate coarse output and standalone merge tensor.

Contract. Fine and coarse normalization must use a common score shift. Density-one routing reduces to the fine path; partially causal centroids are rejected.

`Implementation: experiments/kitchen_attention/native/kitchen_sm75.cu; experiments/kitchen_attention/attention.py`

SPR11 INT8 PV and compact-drop TileLang alternatives

Selection: experimental

Quantize V per leaf/channel and quantize probabilities for an integer PV path; optionally compact drop routes before the existing fine kernel.

Work removed. Trade FP16 PV traffic/work for integer operands and shorten selected-block traversal.

Contract. Probability/V quantization changes the approximation. These flags are retained experiments and are not selected by current integrated Group32 sparse callers.

`Implementation: patches/tilelang_attention_tradeoffs.py; patches/tilelang_sparse_attention.py`

SPR12 Legacy Sol drop preparation fusion

Selection: experimental

Fuse preparation for the retained Sol centering/nearest-even recipe, omit unused V8/coarse/LSE arrays, and bind the prepared geometry to its routes.

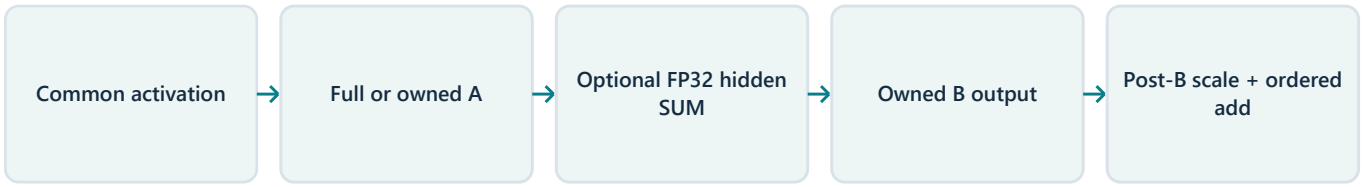
Work removed. Remove legacy drop-only preparation stages without importing scalar-pooling work.

Contract. Preserve this older method own quantization and route equation; it is not interchangeable with the Group32 or native Kitchen sparse recipes.

`Implementation: patches/tilelang_sol_drop_fused.py; benchmarks/attention_sol_drop_study.py`

LOR / Online adapter ownership and arithmetic

For delta = scale * B(Ax), preserve Native adapter order and the post-B scale. Reduce work by changing storage ownership or communication only where the selected numerical contract permits it.



Conceptual dataflow and ownership; independent techniques have their own contracts.

LOR01 Native ordered adapter application

Selection: active

Apply each adapter own A/B products, strength and addition in source order. Activate SwiGLU before gathering the adapter input when the Native operation uses activated features.

Work removed. Provide a stable base for storage/ownership optimizations without rebuilding full dense delta weights online.

Contract. Do not pre-scale B or concatenate adapters implicitly: those change floating GEMM/reduction boundaries.

`Implementation: patches/h3_lora.py; patches/offline_lora_reference.py`

LOR02 Single rank-major input receive buffer

Selection: active

Gather activated rank-owned inputs into one receive storage, then expose the Native feature order to the full-A computation.

Work removed. Avoid a list of per-rank receive tensors, a concatenation copy and simultaneous duplicate gathered inputs.

Contract. Packed gate/up ownership must be reordered component-wise. Keep the backing storage alive only until all views are consumed.

`Implementation: patches/h3_lora.py`

LOR03 Owned adapter output views

Selection: active

Scale and add only the rank-owned B output rows, then release the view and its full backing product before the next adapter.

Work removed. Avoid full-width delta scaling/add buffers and unintended lifetime overlap between adapters.

Contract. A narrow view can retain a full allocation. Explicitly drop both caller and internal references after the ordered add.

`Implementation: patches/h3_lora.py`

LOR04 Selected-B FC1 computation

Selection: active

Load and multiply only the owner half of each gate/up B component for the qualified ordinary DiT FC1 rank128 adapter geometry.

Work removed. Remove unowned FC1 B factor storage and unowned output computation.

Contract. Retain full A and post-B scale/order. Wrong geometry, other adapter ranks and refiner targets use full B.

`Implementation: patches/h3_lora.py`

LOR05 Component-aware selected-B QKV

Selection: active

Pack each owner rows independently from all three QKV B components for the guarded rank128 DiT target. Carry packed representation sizes into admission accounting.

Work removed. Avoid materializing the other owner Q/K/V delta outputs and B rows.

Contract. Keep separate adapter GEMMs and full A. Selection is an H3 shape guard, not a universal LoRA slicing rule.

`Implementation: patches/h3_lora.py; patches/comfy_tp_runtime.py`

LOR06 FP32 Row-A partial hidden reduction

Selection: active

Read only owned A columns, compute A_r x_r locally, FP32-SUM the small low-rank hidden state, and apply full B followed by the original strength.

Work removed. Replace full-width floating input gathering with communication proportional to adapter rank.

Contract. The changed reduction order is approximate relative to one full A GEMM. Keep the hidden SUM in FP32 independently of base partial-sum transport.

`Implementation: patches/h3_lora.py`

LOR07 Bounded hidden-state staging

Selection: active

Limit live gathered/low-rank temporary rows, reuse only the selected staging representation and release per-adapter outputs before advancing.

Work removed. Bound Row-A partial and adapter output peaks for long sequences or multiple adapters.

Contract. Account for real backing tensors, not only view sizes. Preserve adapter ordering across staged work.

`Implementation: patches/h3_lora.py; docs/reports/ROW_PARTIAL_MEMORY.md`

LOR08 Policy-aware factor residency and identity

Selection: active

Validate canonical targets, factor shapes, alpha and finite strengths once; include adapter source identity, owner representation and selected modes in cache keys.

Work removed. Cache only the factor rows/columns required by the selected ownership method and reuse their prepared representation.

Contract. Detect file mutation and reject unconsumed or duplicate targets. Zero-strength adapters need no factor computation.

`Implementation: patches/h3_lora.py; patches/weight_residency.py`

LOR09 Registered-buffer module copies

Selection: active

Represent online adapters and refiner wrappers as modules with registered tensors and immutable rank metadata, copied through the residency mapper.

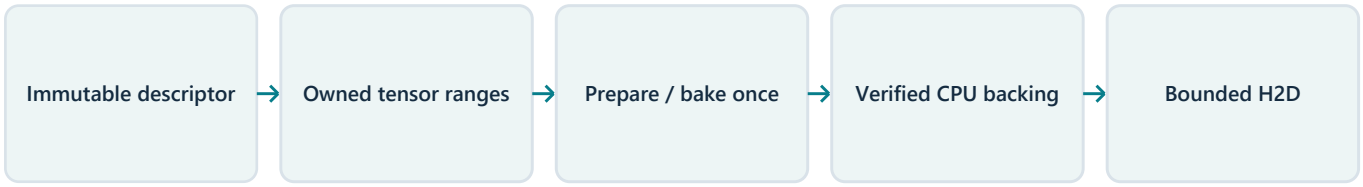
Work removed. Reuse canonical CPU preparation and controlled H2D staging without reconstructing adapter closures for each block.

Contract. A bound method referencing the CPU owner can defeat GPU residency. Copy parameters, buffers and ownership consistently.

`Implementation: patches/h3_lora.py; patches/weight_residency.py`

IO / Offline baking and checkpoint access

Move repeatable work to preparation and read only admitted payloads. Preserve checkpoint representation and immutable identity while replacing full-file reads, repeated validation or repeated adapter merging with bounded owned operations.



Conceptual dataflow and ownership; independent techniques have their own contracts.

IO01 Whole-layer offline adapter baking

Selection: active

Dequantize a complete layer, apply ordered LoRA updates in the reference domain and quantize once into the normal checkpoint representation. Assign whole layers across up to two GPUs.

Work removed. Eliminate recurring online A/B GEMMs and adapter communication during inference.

Contract. Do not split a matrix in a way that changes its quantizer domain. Publish atomically and identify the complete recipe and source stack.

`Implementation: patches/offline_lora.py; patches/offline_lora_reference.py`

IO02 Bounded offline merging into TP2 bundles

Selection: active

For a rank-sharded bundle, reconstruct the full matrix required by the merge, apply the ordered adapter update, requantize, then split the result into the owned component layout.

Work removed. Bound host/device preparation by matrix/block instead of creating an entire merged model in RAM.

Contract. Internal bundle merging must be explicitly enabled for descriptor callers; a path suffix alone cannot prove a checkpoint was already baked.

`Implementation: patches/tp2_bundle.py; patches/comfy_tp_runtime.py`

IO03 Persistent disk-baked rank shards

Selection: active

Bake the owned outputs to immutable rank-block safetensors and publish a descriptor/state manifest after all required blocks are complete. Reopen them through DiskBakedReader.

Work removed. Avoid repeating offline merges on each request, process start or unchanged adapter stack.

Contract. Verify recipe, base identity, adapter strengths, auxiliary tensors and complete state. Partial writes must never become a complete published descriptor.

`Implementation: patches/tp2_disk_bake.py`

IO04 Cached proof of immutable baked state

Selection: active

Validate state schema on every load, but memoize a full file/state proof for an unchanged shard/manifest identity. Recheck the admitted set at block boundaries and addressed files on reads.

Work removed. Avoid repeated expensive full integrity work for unchanged persisted shards.

Contract. Cache proof identities and reader metadata, not merged tensors. Detect mutations and require reload instead of trusting stale evidence.

`Implementation: patches/tp2_disk_bake.py; patches/tp2_bundle.py`

IO05 Positional owned-range reads

Selection: active

Translate safetensors metadata and row/column ownership into exact file byte ranges. Use positional pread where available, otherwise a scoped seek/readinto implementation.

Work removed. Read the owner tensor pieces without loading full matrices or moving a shared file cursor between concurrent readers.

Contract. Validate file identity, bounds, dtype and tensor sizes before constructing views. Platform-specific readers preserve the same bytes.

`Implementation: patches/windowed_tp.py; patches/rank_local_int8.py`

IO06 Contiguous verified rank-block export

Selection: alternative

Export prepared rank-owned weights, replicated scales and required metadata into contiguous safetensors shards plus a verified descriptor.

Work removed. Replace many strided source-range reads and repeated ownership reconstruction with block-local contiguous payloads.

Contract. Retain component-aware ownership and source hashes. Exports change storage layout, not tensor values or upstream checkpoints.

`Implementation: patches/rank_local_int8.py; patches/tp2_bundle.py`

IO07 Bounded full-row and contiguous column reads

Selection: experimental

Allow a bounded full-row scratch read before slicing owned columns. Read full-width column projections contiguously without the row-stride path.

Work removed. Trade additional source bytes for fewer small read calls while capping scratch memory.

Contract. Keep direct owned ranges as the normal path. Row amplification and I/O concurrency are distinct from GPU numerical kernels.

`Implementation: patches/windowed_tp.py; patches/rank_local_int8.py`

IO08 Prepare-once canonical CPU modules

Selection: active

Build canonical CPU modules with policy, LoRA representation and tensor-flow folds once. Cache these prepared owners and copy them through explicit staging leases.

Work removed. Avoid repeated parsing, module construction and nonlinear preparation during recurring layer evaluations.

Contract. Canonical CPU tensors remain immutable under GPU execution. Cache keys include source and complete numerical policy.

`Implementation: patches/windowed_tp.py; patches/host_weight_cache.py; patches/custom_te_loader.py`

IO09 Coalesced TE block backing views

Selection: retired

Read a complete rank-local TE payload sequentially into one CPU backing allocation and expose disjoint tensor views during preparation.

Work removed. Amortize many tiny tensor reads and allocations into a single block read.

Contract. This retained TE loading ablation is not a production path. Views retain the complete payload; count that backing allocation in peak budgets.

`Implementation: benchmarks/historical_te_loading.py`

IO10 Descriptor, mmap and read-advice alternatives

Selection: experimental

Study shared versus independent descriptors, preadv, mmap views, sequential/random advice and bounded additional reader workers as alternative byte-delivery schedules.

Work removed. Trade read-call overhead, page-cache behavior and allocation copies for a fixed tensor-byte workload.

Contract. These are storage-study variants. Do not change the pinned source reader or assume that mmap-backed pages are free resident memory.

`Implementation: benchmarks/te_io_storage_study.py; benchmarks/te_io_pipeline.py`

IO11 Source-only immutable bootstrap overlays

Selection: active

Deliver frozen source by immutable commit/hash and install only exact approved package differences into an isolated overlay, retaining the native matching Torch/CUDA family where authorized.

Work removed. Avoid large wheel/model dataset mounts and repeated transfers of unchanged runtime material.

Contract. This optimizes delivery, not attention arithmetic. Keep core pins and source ABI fixed; authenticate private activations without publishing credentials.

`Implementation: benchmarks/prepare_kaggle_direct_pip.py; experiments/kitchen_attention/dense_delivery.py`

IO12 Hash-bound source deltas and lazy payload delivery

Selection: active

Apply a verified small source delta over an immutable retained distribution, reconstruct its capsule hash, and fetch learned block payloads only for the bounded work that needs them.

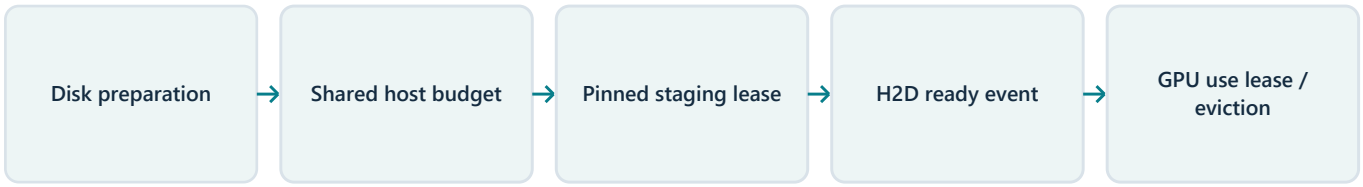
Work removed. Reduce repeated full source/archive transfers and avoid loading unused model payloads.

Contract. Record the reconstructed source identity and preserve historical archives. Source deltas cannot silently replace frozen ComfyUI or unpin core operators.

`Implementation: experiments/kitchen_attention/delivery.py; benchmarks/build_kaggle_light_delivery.py`

RES / Host cache, GPU windows and phase scheduling

Treat disk construction, pageable cache, pinned staging and GPU residency as distinct owners under explicit byte budgets. Prefetch is useful only while it respects foreground work and the next actual use.



Conceptual dataflow and ownership; independent techniques have their own contracts.

RES01 Process-shared host admission

Selection: active

Pass one synchronized host budget to both spawned ranks. Reserve bytes for pageable modules, preparation and pinned staging, with system-memory headroom.

Work removed. Avoid multiplying the nominal RAM allowance by rank or ignoring simultaneous construction and staging allocations.

Contract. A tensor view is charged by its retained backing. Denied speculative work must not strand a reservation.

`Implementation: patches/host_weight_cache.py`

RES02 Pageable cache with bounded pin staging

Selection: active

Keep reusable prepared modules in a pageable cache. Pin only the admitted transfer representation while its staging lease is active, then release it after H2D completion.

Work removed. Avoid permanently pinning the entire CPU cache and duplicate long-lived pinned owners.

Contract. Pageable, pin-construction and transfer lifetimes are separately accounted. A nonblocking transfer alone does not prove its source can be freed.

`Implementation: patches/host_weight_cache.py; patches/weight_residency.py`

RES03 Foreground-priority deadline prefetch

Selection: active

Queue CPU preparation by next-use deadline and reuse value, with one worker per cache and foreground demand admitted ahead of speculative modules at safe boundaries.

Work removed. Overlap useful disk/CPU preparation with GPU computation without allowing speculative work to block imminent layers.

Contract. Changing priority does not cancel an already executing unsafe operation. Scope cancellation at module/preparation boundaries.

`Implementation: patches/host_weight_cache.py`

RES04 Reuse-aware host eviction

Selection: active

Choose cached owners for eviction using recency and planned next use, preserve useful repeated-evaluation groups, and release their budget reservations explicitly.

Work removed. Avoid discarding a soon-reused DiT block merely because a speculative TE block arrived.

Contract. Evict only unleased owners. Identity and reuse plans must be reset when a request/policy changes.

`Implementation: patches/host_weight_cache.py; patches/weight_residency.py`

RES05 Event-owned H2D residency windows

Selection: active

Prefetch admitted modules on the transfer stream, record completion events, and wait on those events at use. Associate every resident module with byte, preparation and use owners.

Work removed. Overlap transfers with computation while bounding live GPU weights and staging storage.

Contract. Ready, in-flight and leased states differ. Do not evict an in-use module or reuse an unfinished transfer buffer.

`Implementation: patches/weight_residency.py`

RES06 Next-use GPU eviction and explicit leases

Selection: active

Track remaining uses, evict unleased far-next-use groups when making room, and retain currently leased attention/MLP/auxiliary modules until their operation ends.

Work removed. Use a bounded GPU window without reloading every group immediately after use or evicting active work.

Contract. Admission accounts for pending allocations and external reservations. Completion and exception paths release leases deterministically.

`Implementation: patches/weight_residency.py`

RES07 Adaptive one/two-group lookahead

Selection: active

Choose supported lookahead depth from available VRAM, activation reserve, group sizes and current in-flight residency. Reduce speculation under pressure.

Work removed. Prefetch an additional useful group when safe without reserving a fixed large window on every shape.

Contract. Lookahead is byte-bounded and distinct from CPU cache depth. Larger activation sequences can force a smaller safe window.

`Implementation: patches/weight_residency.py`

RES08 Linux cgroup and PSI-aware admission

Selection: active

Read available cgroup-v2 charge, nonfile/file-cache usage and pressure stall information. Protect useful file cache and cap concurrent preparation/speculation before exhausting the effective host limit.

Work removed. Avoid reclaim/thrashing caused by treating nominal machine RAM as the only memory constraint.

Contract. Metrics are read-only and optional. Missing metrics retain the explicit managed/system-RAM budget rather than inventing zero pressure.

`Implementation: patches/host_memory_pressure.py; patches/host_weight_cache.py`

RES09 Separated startup groups

Selection: active

Prepare vision and embedding/startup modules as independent CPU groups rather than an undifferentiated model payload.

Work removed. Allow only the next required startup owner to occupy a GPU window or pinned stage.

Contract. Preserve Native embedding and vision execution order and canonical CPU backing.

`Implementation: patches/comfy_tp_runtime.py; patches/comfy_tp_models.py`

RES10 On-demand vision GPU lifetime

Selection: active

Copy the vision owner to its device only for the actual frontend work, fence completion and release its GPU tensors before admitting the next phase.

Work removed. Avoid retaining the vision tower through text layers or diffusion sampling.

Contract. A synchronization fence is part of ownership. Full frontend replication remains a separate property from where weight groups reside.

`Implementation: patches/comfy_tp_models.py`

RES11 Rank-zero auxiliary projection leases

Selection: active

Lease refiner and input/output auxiliary modules on rank zero, compute the required Native interfaces and broadcast shared results to both ranks.

Work removed. Avoid replicating auxiliary learned work and weights that are not tensor-parallel block computation.

Contract. Keep public conditioning, refiner normalization and layout semantics. Only the explicitly owned auxiliary work is rank-zero-only.

`Implementation: patches/windowed_tp.py; patches/comfy_tp_runtime.py`

RES12 Retain DiT groups between real evaluations

Selection: active

Extend next-use plans over the actual sequence of denoiser evaluations, retaining useful groups rather than treating each call as the end of model lifetime.

Work removed. Reuse expensive prepared/resident diffusion weights across sampler steps.

Contract. Use actual command/evaluation boundaries rather than a guessed timestep count, and surrender residency when another phase needs the budget.

`Implementation: patches/comfy_tp_runtime.py`

RES13 Cross-model CPU preparation

Selection: active

When TE completes, prepare the known next DiT CPU groups while the parent completes conditioning/packing. Preserve separate CPU and GPU admission.

Work removed. Move deterministic next-phase CPU loading off the immediate DiT start path.

Contract. Cross-model CPU prefetch does not imply early DiT GPU residency or permission to exceed host preparation limits.

`Implementation: patches/comfy_tp_runtime.py`

RES14 Decode-start next-request preparation

Selection: active

At output decode start, schedule reusable TE startup groups and a bounded lifecycle-controlled prefix for a compatible next request.

Work removed. Use decode time for preparation that can be reused after the current sampling phase.

Contract. Speculation obeys foreground decode and memory-pressure ownership. It does not preload an arbitrary next prompt activation.

`Implementation: patches/comfy_tp_runtime.py`

RES15 Idle compatible-prefix residency

Selection: alternative

After prompt compatibility checks, load only required parent reference encoders and a bounded worker TE prefix during an admitted idle interval.

Work removed. Amortize compatible next-request startup without making idle residency mandatory.

Contract. Idle is explicit and off by default. Cancellation, incompatible prompts and foreground work revoke speculation at defined boundaries.

`Implementation: patches/comfy_idle_lifecycle.py; patches/comfy_tp_runtime.py`

RES16 Event-owned activation handoff

Selection: active

Copy a boundary activation through a dedicated handoff owner with completion tracking, and close that owner when the receiving stage is finished.

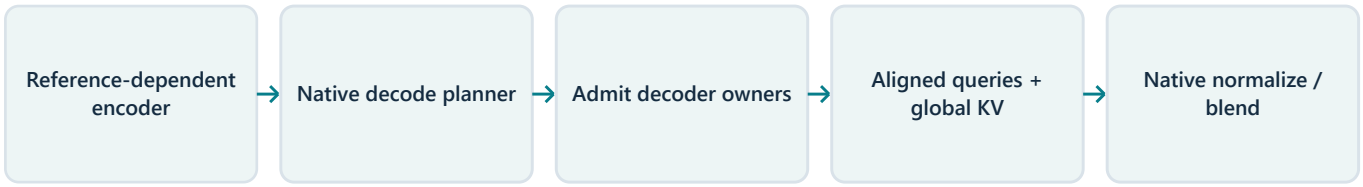
Work removed. Avoid accidental duplicate cross-phase activation residency and synchronize only the required handoff.

Contract. Keep source and destination alive until stream work completes; a pipe acknowledgement does not by itself complete a CUDA transfer.

`Implementation: patches/weight_residency.py`

VAE / Component loading and parallel decode

Keep Native temporal/spatial planning, normalization and blending in the public interface. Parallelize only the declared learned work, with coordinated admission before distributed execution and a scoped fallback owner.



Conceptual dataflow and ownership; independent techniques have their own contracts.

VAE01 Metadata-only staged VAE shells

Selection: active

Construct component metadata without reading learned tensors. Instantiate/load only the encoder required by reference input and load the decoder at output time.

Work removed. Avoid unconditional video/audio encoder and decoder payload reads and simultaneous component residency.

Contract. Retain required normalization buffers and the frozen constructor/forward API. Decoder-first execution must work without an encoder preload.

`Implementation: patches/staged_vae.py; patches/reference_loaders.py`

VAE02 Reference-demand encoder planning

Selection: active

Derive immutable hints from graph-required image/video/audio references and pass only those encoder needs into idle/startup loading.

Work removed. Avoid speculative encoder construction for absent reference modalities.

Contract. Hints identify component demand, not activation values. Actual reference encoding still selects the Native public encoder path.

`Implementation: patches/h3_reference_plan.py; patches/comfy_sampler.py`

VAE03 Independent decoder memory admission

Selection: active

Separate Video decoder weight and activation reserve budgets from the DiT weight window. Coordinate room/refusal on both ranks before starting decoder collectives.

Work removed. Prevent a narrow DiT window policy from incorrectly excluding or over-admitting a replicated decoder phase.

Contract. Count learned decoder replicas and external activation headroom. Both owners must make the same distributed execution decision.

`Implementation: patches/comfy_tp_runtime.py; patches/weight_residency.py; patches/parallel_vae.py`

VAE04 Weight-free Native parent decode planner

Selection: active

Build a metadata-only Native Video planner with tiny normalization buffers and a remote decode-pixels callback. Keep learned decoder layers only in admitted workers.

Work removed. Avoid a redundant learned parent decoder and its CPU/GPU weight lease during remote decoding.

Contract. Preserve Native chunking, interpolation, blending and output writes. Restore the original patcher/model owner in a scoped finally path.

`Implementation: patches/video_decode_planner.py; patches/staged_vae.py`

VAE05 Global-KV query-parallel Video attention

Selection: active

Split query ranges on the retained 128-query scale-group boundary while preserving global K/V and required full-sequence inputs. Exchange outputs in Native token order.

Work removed. Share query-side attention and downstream owned-token work between two decoder replicas.

Contract. Global KV/quantization context must not be replaced by cropped spatial tiles. Check the trained legacy decoder shape/API and aligned range availability.

`Implementation: patches/parallel_vae.py`

VAE06 Owned final normalization and projection

Selection: active

Apply the decoder final norm and proj_out to each owned query/token range, then exchange the completed output in the required layout.

Work removed. Avoid reconstructing and projecting the complete final activation on both replicas.

Contract. Changing matrix row geometry can change backend rounding. Preserve declared numerical/output contracts and Native global planning.

`Implementation: patches/parallel_vae.py`

VAE07 Coordinated Native fallback and scoped leases

Selection: active

Resolve unsupported geometry, unavailable ABI and admission refusal before collectives. Exit remote planner/decoder leases and invoke the preserved Native decode owner.

Work removed. Provide a bounded supported path without retaining both a remote and parent learned GPU decoder.

Contract. One rank cannot silently fall back while the other enters collectives. Restore patcher state and release in-flight ownership on exceptions.

`Implementation: patches/video_decode_planner.py; patches/parallel_vae.py; patches/comfy_sampler.py`

VAE08 Native public tiling and chunk normalization

Selection: active

Delegate temporal/spatial chunk coordinates, overlap fades, normalization buffers and final assembly to the frozen public VAE API around the selected learned decoder.

Work removed. Reuse the existing bounded decode planner instead of retaining an extra full-resolution intermediate pipeline.

Contract. Preserve crop coordinates and temporal context. A raw decode fragment is not a complete substitute for the public normalized output.

`Implementation: patches/staged_vae.py; patches/comfy_sampler.py; patches/parallel_vae.py`

VAE09 Single-owner FP32 Audio decoding

Selection: active

Load the required audio decoder on one GPU and run the complete Native FP32 stereo/chunk pipeline through the public API.

Work removed. Avoid an additional audio decoder replica, channel collectives and alternative temporal reconstruction.

Contract. The runtime selects Native single-GPU audio even when an old workflow requests an obsolete automatic parallel mode.

`Implementation: patches/comfy_sampler.py; patches/staged_vae.py`

VAE10 Overlapping spatial slab decoding

Selection: historical

Decode two independently loaded spatial slabs with explicit latent overlap, map them to output coordinates and blend the overlap.

Work removed. Bound each decoder spatial workload and distribute independent slab work.

Contract. Cropping changes global attention context and is an approximation. Keep latent/output scale ratios and fade coordinates explicit; this is not the current query-parallel route.

`Implementation: patches/slab_video_vae.py; patches/multi_gpu_video_vae.py`

VAE11 Video tensor/sequence-parallel block alternatives

Selection: historical

Partition decoder projection weights and row partials, or partition token-boundary work while gathering the global context needed by the block.

Work removed. Distribute learned matrix work or activations instead of replicating every decoder block.

Contract. Keep quantizer grouping, global norm/attention context and legacy block arguments. These retained adapters are distinct from the active replicated-decoder query split.

`Implementation: patches/tp_video_vae.py`

VAE12 Stereo channel-parallel Audio decoding

Selection: retired

Assign one stereo latent channel to each persistent rank decoder, decode its mono view in FP32 and gather the resulting channels in original order.

Work removed. Distribute left/right learned decode work and output storage.

Contract. Require stereo geometry and account for shape-dependent normalization/GEMM rounding. This retained technique is removed from normal runtime dispatch.

`Implementation: patches/audio_channel_decode.py; patches/multi_gpu_audio_vae.py`

VAE13 Temporal and hybrid Audio tiling

Selection: historical

Distribute overlapping temporal chunks across decoders, cross-fade reconstructed waveforms, and optionally combine temporal ownership with stereo-channel ownership.

Work removed. Parallelize chunked audio work and bound each chunk activation footprint.

Contract. Preserve temporal context, normalization and sample-aligned fades. These historical manager paths are not the Native audio default.

`Implementation: patches/multi_gpu_audio_vae.py`

ALT / Retained alternative optimization methods

Retired and experimental methods remain part of the technique inventory. Their mechanisms are described here without treating an old result or a surviving function as a current deployment selection.



Conceptual dataflow and ownership; independent techniques have their own contracts.

ALT01 Two-stage layer pipeline parallelism

Selection: historical

Place contiguous layer ranges on two GPUs and transfer the boundary activation once per model pass. Preserve mask, positions, KV/intermediate state and Native model ordering across the boundary.

Work removed. Fit separate layer weight sets without a collective at every row projection.

Contract. A single stream traverses stages serially; this is a different weight/communication schedule from TP2, not simultaneous execution of both layer halves.

`Implementation: patches/pp_dit.py; patches/pp_qwen3vl.py`

ALT02 Sequence-sharded block boundaries

Selection: historical

Gather Native floating activations from local token ranges, execute the repaired tensor-parallel block with global RoPE/modulation/masks, then return the owner token slice.

Work removed. Store only local sequence slices at block boundaries while retaining the existing TP arithmetic inside.

Contract. The repaired adapters gather activations, not legacy INT8 sequence payloads. Global attention context and modulation cannot be inferred from one local slice.

`Implementation: patches/sp_int8_dit.py; patches/sp_int8_qwen3vl.py`

ALT03 Groupwise packed INT8 partial transport

Selection: experimental

Quantize groups of INT32 partial outputs to signed byte codes plus serialized FP32 group scales, gather owner payloads and reconstruct/add them before the output epilogue.

Work removed. Trade a smaller byte payload for group metadata and encode/decode work.

Contract. Separate rank scales require reconstruction before addition; byte-code SUM would overflow or apply incompatible scales. This is approximate and not default transport.

`Implementation: patches/tilelang_transport_tradeoffs.py; patches/quantized_collectives.py`

ALT04 Floating partial reduction alternative

Selection: retired

Convert integer GEMM partials to floating point before distributed SUM instead of preserving the exact INT32 collective or the bounded scaled-FP16 codec.

Work removed. Permit a floating collective representation and different communication kernel selection.

Contract. Conversion and floating SUM change cancellation/reduction behavior. This retired control does not override the current policy-selected transport.

`Implementation: docs/reports/TP2_P2_OPTIMIZATION_REPORT.md; patches/tilelang_transport_tradeoffs.py`

ALT05 Concatenated LoRA A projections

Selection: retired

Stack multiple adapter A factors into one projection, split its low-rank outputs and apply the corresponding B products and post-B strengths.

Work removed. Amortize repeated A input reads and GEMM setup across an adapter stack.

Contract. Changing the A GEMM shape changes rounding and staging. This older option is rejected by current runtime controls; separate ordered adapters remain the supported method.

`Implementation: benchmarks/h3_lora_options_audit.py; docs/reports/ONLINE_LORA_OPTIONS.md`

ALT06 Streamed SwiGLU row preparation

Selection: retired

Produce/rotate/quantize activation chunks in a bounded schedule instead of retaining a complete branch product and its full rotated representation.

Work removed. Trade recomputation and chunk orchestration for shorter temporary lifetimes.

Contract. The old streamed control is retired. The supported fused_recompute implementation is a separate, explicitly defined FP32 preparation path.

`Implementation: benchmarks/qualify_streamed_swiglu.py; docs/reports/FC2_QUANTIZER_MEMORY.md`

ALT07 Independent lossless ZSTD tensor frames

Selection: experimental

Encode bounded chunks within each tensor/rank as independent ZSTD frames and retain a side index. Decode frames through a bounded outer worker pool.

Work removed. Reduce stored/transferred checkpoint bytes while permitting random frame-level reconstruction.

Contract. This CPU codec is not integrated into the active loader. Bound output allocation and context/thread counts; decompression alone does not include disk or tensor assembly.

`Implementation: benchmarks/te_compression_local.py; docs/reports/TE_COMPRESSION_LOCAL_20261020.md`

ALT08 BitShuffle plus block compression

Selection: experimental

Transpose bit planes within bounded blocks, then apply the selected Blosc/ZSTD compressor, retaining independently decodable tensor/rank frames.

Work removed. Expose repeated bit-plane patterns to the entropy codec and enable bounded parallel decoding.

Contract. Keep dtype typesize and filter order explicit. Bit-plane structure is a property of the encoded data, not a guarantee of compression for every INT8 tensor.

`Implementation: benchmarks/te_compression_local.py`

ALT09 Direct and ZigZag FSE byte coding

Selection: experimental

Entropy-code tensor bytes with FSE directly, or map signed values through ZigZag before the same framed coding procedure.

Work removed. Compress nonuniform byte distributions with a table-based entropy representation.

Contract. Retain a reversible frame/index format and actual decoder output allocation. Direct and ZigZag variants are separate codecs, not interchangeable stored payloads.

`Implementation: benchmarks/te_compression_local.py; benchmarks/build_te_compression_codecs.ps1`

ALT10 Typed ZipNN codec path

Selection: experimental

Use the codec typed-tensor entry point rather than treating an arbitrary byte buffer as an assumed supported INT8 numerical format.

Work removed. Apply specialized lossless representation coding where the library supports the tensor dtype.

Contract. Keep unsupported dtype handling explicit; a proposed or blocked codec is not an active H3 decompression stage.

`Implementation: benchmarks/te_compression_local.py; benchmarks/requirements-te-compression.txt`

ALT11 Codec frame-size and worker scheduling

Selection: experimental

Choose independently decodable frame sizes and a bounded number of outer decode workers, with one library thread per frame job. Release materialized outputs when consumed.

Work removed. Balance frame/index overhead, CPU parallelism, allocation peaks and random-read granularity.

Contract. Avoid nested oversubscription and count live decompressed buffers. This scheduling method does not change quantized tensor values.

`Implementation: benchmarks/te_compression_local.py`

ALT12 Folded-strength concatenated low-rank updates

Selection: retired

Stack adapter A factors vertically and combine strength-scaled B factors horizontally, representing the sum of several updates as a composed larger low-rank operator.

Work removed. Amortize A/B projection setup and repeated activation reads across multiple adapters.

Contract. Pre-folding strengths and changing GEMM/reduction shapes changes floating arithmetic. This historical fused-B variant is distinct from supported ordered post-B scaling.

`Implementation: docs/reports/H3_INT8_SWIGLU_LORA.md; docs/reports/ONLINE_LORA_NATIVE_ORDER_REPAIR.md`

ALT13 Quantized sequence activation exchange

Selection: historical

Exchange INT8 activation codes together with the scales and ownership metadata needed to reconstruct a global sequence representation before the consuming operation.

Work removed. Reduce exchanged activation payload relative to a full floating-point gather.

Contract. Quantizer domains, global normalization and rotary context must remain coherent. The old sequence-payload variant is not the repaired adapter that gathers Native activations.

`Implementation: docs/reports/TEXT_ENCODER_SP_INT8_REPORT.md; patches/sp_int8_qwen3vl.py`

Core equations

Shared activation scale

global_max[m] = MAX_r(local_max_r[m]); scale[m] = max(global_max[m] / 127, floor). Both owners quantize against the same row scale.

Row projection

Y = dequantize(SUM_r(INT8_GEMM_r), row_scale, weight_scale) + bias. Scaled FP16 transport deliberately approximates the integer SUM.

Omitted-leaf pooling

coarse_logit[i,b] = dot(q[i], mean_k[b]) / sqrt(D) + log(valid_count[b]). Merge omitted centroids and selected fine tokens using a common online normalization state.

Row-A LoRA

hidden = SUM_r(A_r x_r) in FP32; delta = strength * B hidden. Apply separate adapters in their original order.

Residual gauge

y = x / c; normalized epsilon = eps / c^2. Fold branch output division before the storage cast; restore public unscaled outputs.

Implementation source catalogue

Every listed path is linked to one or more technique IDs and included in the embedded source-hash manifest. Historical report sources describe retained methods whose current runtime selection is retired.

`benchmarks/attention_grouped_study.py`

Techniques: ATT21

`benchmarks/attention_sol_drop_study.py`

Techniques: SPR12

`benchmarks/bounded_column_int8.py`

Techniques: LIN17

`benchmarks/build_kaggle_light_delivery.py`

Techniques: IO12

`benchmarks/build_te_compression_codecs.ps1`

Techniques: ALT09

`benchmarks/h3_lora_options_audit.py`

Techniques: ALT05

`benchmarks/historical_te_loading.py`

Techniques: IO09

`benchmarks/prepare_kaggle_direct_pip.py`

Techniques: IO11

`benchmarks/qualify_streamed_swiglu.py`

Techniques: ALT06

`benchmarks/requirements-te-compression.txt`

Techniques: ALT10

`benchmarks/te_compression_local.py`

Techniques: ALT07, ALT08, ALT09, ALT10, ALT11

`benchmarks/te_io_pipeline.py`

Techniques: IO10

`benchmarks/te_io_storage_study.py`

Techniques: IO10

`custom_nodes/minimax_h3_tp2/__init__.py`

Techniques: SYS07

`docs/reports/FC2_QUANTIZER_MEMORY.md`

Techniques: ALT06

`docs/reports/H3_INT8_SWIGLU_LORA.md`

Techniques: ALT12

`docs/reports/ONLINE_LORA_NATIVE_ORDER_REPAIR.md`

Techniques: ALT12

`docs/reports/ONLINE_LORA_OPTIONS.md`

Techniques: ALT05

`docs/reports/ROW_PARTIAL_MEMORY.md`

Techniques: LOR07

`docs/reports/TEXT_ENCODER_SP_INT8_REPORT.md`

Techniques: ALT13

`docs/reports/TE_COMPRESSION_LOCAL_20261020.md`

Techniques: ALT07

`docs/reports/TP2_P2_OPTIMIZATION_REPORT.md`

Techniques: ALT04

`experiments/kitchen_attention/attention.py`

Techniques: ATT01, ATT13, ATT14, ATT17, SPR08, SPR09, SPR10

`experiments/kitchen_attention/build.py`

Techniques: ATT20

`experiments/kitchen_attention/delivery.py`

Techniques: IO12

`experiments/kitchen_attention/dense_delivery.py`

Techniques: IO11

`experiments/kitchen_attention/derive.py`

Techniques: ATT15, ATT16

`experiments/kitchen_attention/graph.py`

Techniques: ATT18

`experiments/kitchen_attention/native/h3_dense_utils.cuh`

Techniques: ATT16

`experiments/kitchen_attention/native/kitchen_sm75.cu`

Techniques: ATT14, ATT15, ATT17, SPR07, SPR09, SPR10

`experiments/kitchen_attention/native/routed_loop.cuh`

Techniques: SPR08

`experiments/kitchen_attention/native/vendor/attn_utils.cuh`

Techniques: ATT01

`experiments/kitchen_attention/te_precision.py`

Techniques: ATT13

`patches/audio_channel_decode.py`

Techniques: VAE12

`patches/comfy_idle_lifecycle.py`

Techniques: RES15

`patches/comfy_sampler.py`

Techniques: SYS02, SYS03, VAE02, VAE07, VAE08, VAE09

`patches/comfy_tp_models.py`

Techniques: RES09, RES10

`patches/comfy_tp_runtime.py`

Techniques: SYS02, SYS03, SYS04, SYS05, SYS06, SYS08, NUM01, NUM07, LOR05, IO02, RES09, RES11, RES12, RES13, RES14, RES15, VAE03

`patches/custom_te_loader.py`

Techniques: IO08

`patches/diagnostic_metrics.py`

Techniques: SYS08

`patches/dit_column_linear.py`

Techniques: LIN14, LIN15, LIN16

`patches/h3_configuration.py`

Techniques: SYS07

`patches/h3_lora.py`

Techniques: NUM04, NUM05, LOR01, LOR02, LOR03, LOR04, LOR05, LOR06, LOR07, LOR08, LOR09

`patches/h3_operator_dependencies.py`

Techniques: SYS07

`patches/h3_operator_policy.py`

Techniques: SYS06, LIN06, LIN08, NUM06, ATT01, ATT02, ATT03, SPR02, SPR04, SPR06

`patches/h3_reference_plan.py`

Techniques: VAE02

`patches/h3_tensor_flow.py`

Techniques: NUM01, NUM02, NUM03, NUM04, NUM07

`patches/host_memory_pressure.py`

Techniques: RES08

`patches/host_weight_cache.py`

Techniques: IO08, RES01, RES02, RES03, RES04, RES08

`patches/kitchen_te_attention.py`

Techniques: ATT02, ATT19, ATT20

`patches/legacy_runtime.py`

Techniques: SYS07

`patches/multi_gpu_audio_vae.py`

Techniques: VAE12, VAE13

`patches/multi_gpu_video_vae.py`

Techniques: VAE10

`patches/offline_lora.py`

Techniques: IO01

`patches/offline_lora_reference.py`

Techniques: LOR01, IO01

`patches/optimized_tp.py`

Techniques: LIN01, LIN02, LIN10, LIN13

`patches/parallel_int8.py`

Techniques: LIN01, LIN10, LIN12, LIN13

`patches/parallel_vae.py`

Techniques: VAE03, VAE05, VAE06, VAE07, VAE08

`patches/pp_dit.py`

Techniques: ALT01

`patches/pp_qwen3vl.py`

Techniques: ALT01

`patches/quantized_collectives.py`

Techniques: SYS05, ALT03

`patches/rank_local_int8.py`

Techniques: SYS01, IO05, IO06, IO07

`patches/reference_loaders.py`

Techniques: VAE01

`patches/runtime_diagnostics.py`

Techniques: SYS08

`patches/slab_video_vae.py`

Techniques: VAE10

`patches/sp_int8_dit.py`

Techniques: ALT02

`patches/sp_int8_qwen3vl.py`

Techniques: ALT02, ALT13

`patches/staged_vae.py`

Techniques: VAE01, VAE04, VAE08, VAE09

`patches/tilelang_attention_p2.py`

Techniques: SPR06

`patches/tilelang_attention_tradeoffs.py`

Techniques: ATT09, ATT10, ATT11, SPR07, SPR11

`patches/tilelang_column_epilogue.py`

Techniques: LIN15

`patches/tilelang_dense_preparation.py`

Techniques: ATT07, ATT08

`patches/tilelang_group32_masked.py`

Techniques: ATT05, ATT11

`patches/tilelang_grouped_fused.py`

Techniques: ATT06

`patches/tilelang_grouped_qk.py`

Techniques: ATT04, ATT21

`patches/tilelang_grouped_tp.py`

Techniques: ATT04

`patches/tilelang_grouped_tp_kitchen.py`

Techniques: ATT04

`patches/tilelang_grouped_tp_kitchen_v2.py`

Techniques: ATT06, ATT09, ATT10, ATT22

`patches/tilelang_grouped_tp_optimized.py`

Techniques: ATT08, SPR01, SPR03, SPR04, SPR05

`patches/tilelang_row_quant.py`

Techniques: LIN02, LIN03, LIN04, LIN05

`patches/tilelang_sage_preparation.py`

Techniques: ATT07, ATT12

`patches/tilelang_sage_qk.py`

Techniques: ATT12

`patches/tilelang_scalar_pool.py`

Techniques: SPR05

`patches/tilelang_sol_drop_fused.py`

Techniques: SPR03, SPR12

`patches/tilelang_sparse_attention.py`

Techniques: SPR01, SPR11

`patches/tilelang_swiglu_pad.py`

Techniques: LIN05, LIN06, LIN07, LIN10

`patches/tilelang_swiglu_quant.py`

Techniques: LIN08, LIN09

`patches/tilelang_transport_p2.py`

Techniques: LIN11, LIN12

`patches/tilelang_transport_tradeoffs.py`

Techniques: LIN11, ALT03, ALT04

`patches/tp2_bundle.py`

Techniques: IO02, IO04, IO06

`patches/tp2_disk_bake.py`

Techniques: IO03, IO04

`patches/tp_dit.py`

Techniques: SYS01, NUM03, NUM05, NUM06

`patches/tp_qwen3vl.py`

Techniques: SYS01, SYS09, LIN16, NUM03, NUM05, NUM06

`patches/tp_video_vae.py`

Techniques: VAE11

`patches/video_decode_planner.py`

Techniques: VAE04, VAE07

`patches/weight_residency.py`

Techniques: SYS06, LOR08, LOR09, RES02, RES04, RES05, RES06, RES07, RES16, VAE03

`patches/windowed_tp.py`

Techniques: IO05, IO07, IO08, RES11